Term Project Report

Computer Networks (SWE3022_41)

Handover Analysis in 5G Networks

Submitted By:

Cherukuri Revanth Sai (2025319046)

To:

Prof Jaehoon (Paul) Jeong

Abstract

The transition to 5G New Radio (NR) networks promises unprecedented bandwidth and low latency, yet the performance of the transport layer remains the bottleneck in high-mobility, multi-user scenarios. This project conducts a comprehensive simulation study using OMNeT++ (v6.2.0) and Simu5G (v1.4.0) to analyze the behavior of three distinct transport protocols—User Datagram Protocol (UDP), Transmission Control Protocol (TCP), and Quick UDP Internet Connections (QUIC)—during handover events in a congested multi-cell environment.

By developing custom application-layer modules and executing a parameter sweep across 270 unique simulation runs, we successfully identified the network saturation point (approximately 60 UEs) and quantified the trade-offs between protocol reliability and overhead efficiency. A key contribution of this report is the explicit analysis of the "Stability Regions" observed in the results. We analyze why performance metrics exhibit a "flatline" behavior at low user counts (due to resource abundance) and why throughput plateaus at high payload sizes (due to the vanishing impact of header overhead). Furthermore, we document the data engineering pipeline developed to merge over 16,000 raw data logs into a single, unified dataset (final_aggregated_results.csv), enabling precise comparative analysis. The results demonstrate that while UDP offers superior latency profiles under low load, reliable protocols like TCP and QUIC are essential for maintaining data integrity during the instability of 5G handovers.

Table of Contents

Chapter 1: Introduction

1.1 Background and Motivation

The telecommunications industry is currently undergoing a massive paradigm shift with the deployment of 5G Standalone (SA) networks. Unlike previous generations, 5G is designed not just for higher speeds, but for massive connectivity density—supporting up to one million devices per square kilometer. However, the physical radio layer is only half the story. The "Transport Layer" of the OSI model, responsible for end-to-end data delivery, faces unique challenges in 5G.

High-frequency mmWave signals are susceptible to blockage, and user mobility necessitates frequent "handovers" (switching connection from one Base Station to another). These physical disruptions can confuse traditional transport protocols. For instance, TCP often mistakes a handover delay for network congestion, aggressively cutting its transmission speed. This project was motivated by the need to understand exactly how legacy protocols (TCP/UDP) and modern protocols (QUIC) react to these specific 5G stressors.

1.2 Problem Statement

Evaluating network performance is often treated as a "black box" exercise—data goes in, and data comes out. However, to optimize 5G networks, engineers must understand the internal dynamics of congestion.

- **Scalability Issue:** At what specific user density does the Quality of Service (QoS) degrade?
- **Protocol Overhead:** Does the size of the data packet significantly impact the total throughput efficiency?
- **Reliability vs. Latency:** In a handover scenario, is it better to drop packets (UDP) or pause to retransmit them (TCP)?

1.3 Project Objectives

This project aims to bridge the gap between theoretical protocol definitions and practical network behavior through simulation. The specific objectives are:

1. **Develop Custom Application Modules:** To write C++ code for OMNeT++ that can generate traffic and measure One-Way Delay (OWD) and Jitter, bypassing the limitations of standard traffic generators.

2. **Analyze Congestion Points:** To determine the "Knee Point" of the network—the exact number of User Equipments (UEs) where the cell capacity saturates.
3. **Evaluate Protocol Efficiency:** To quantify the impact of packet overhead by sweeping payload sizes from 100 Bytes to 1000 Bytes.
4. **Solve Big Data Challenges:** To implement an automated pipeline for processing the 16,000+ data logs generated by the simulation.

Chapter 2: Theoretical Framework

2.1 5G New Radio (NR) Architecture

The simulation is built upon the **Simu5G** framework, which models the 3GPP Release 15/16 specifications.

- **User Equipment (UE):** The mobile device. In our simulation, UEs utilize the NR (New Radio) interface for data transmission.
- **gNodeB (gNB):** The 5G Base Station. Unlike 4G eNodeBs, gNBs in Simu5G support flexible numerologies and Time Division Duplexing (TDD). The scheduler allocates "Resource Blocks" (RBs) to users based on channel quality.
- **The Core:** We utilize a simplified 5G core network to handle IP routing, allowing us to focus primarily on the Radio Access Network (RAN) performance.

2.2 The Mechanics of Handover

Handover is the defining feature of cellular networks. In this project, we simulate an **X2/Xn-based Handover**.

1. **Measurement:** The UE continuously measures the Reference Signal Received Power (RSRP) of neighboring gNBs.
2. **Reporting:** When a neighbor's signal exceeds the current gNB's signal by a hysteresis margin (A3 event), the UE sends a Measurement Report.
3. **Decision:** The source gNB decides to hand over the UE to the target gNB.
4. **Execution:** The data path is switched. During this switching interval (often 20-50ms), packets can be lost or buffered. This interval provides the critical stress test for our transport protocols.

2.3 Transport Layer Protocols: A Comparative Review

2.3.1 UDP (User Datagram Protocol)

UDP is a connectionless, "best-effort" protocol. It has no handshake, no retransmission, and no congestion control.

- **Pros:** Lowest possible latency; minimal header overhead (8 bytes).
- **Cons:** No guarantee of delivery. In a 5G handover, if a packet is dropped, it is gone forever.
- **Relevance:** Used for VoIP and real-time video where speed > accuracy.

2.3.2 TCP (Transmission Control Protocol)

TCP provides reliable, ordered, and error-checked delivery. It uses a "Three-Way Handshake" (SYN, SYN-ACK, ACK) to establish a connection.

- **Congestion Control:** TCP algorithms (like Cubic or Reno) reduce the sending rate if packet loss is detected.
- **Relevance:** The standard for web browsing and file transfers. However, TCP's aggressive back-off during handover can cause noticeable drops in throughput.

2.3.3 QUIC (Quick UDP Internet Connections)

Developed by Google, QUIC runs on top of UDP but implements reliability and congestion control in the user space (application layer).

- **Key Feature:** It eliminates "Head-of-Line" (HOL) blocking. If one packet is lost, it doesn't stop the processing of subsequent packets.
- **Handshake:** QUIC combines the crypto and transport handshake, establishing connections faster than TCP+TLS.
- **Relevance:** Now the backbone of HTTP/3, QUIC is theoretically better suited for "lossy" wireless environments than TCP.

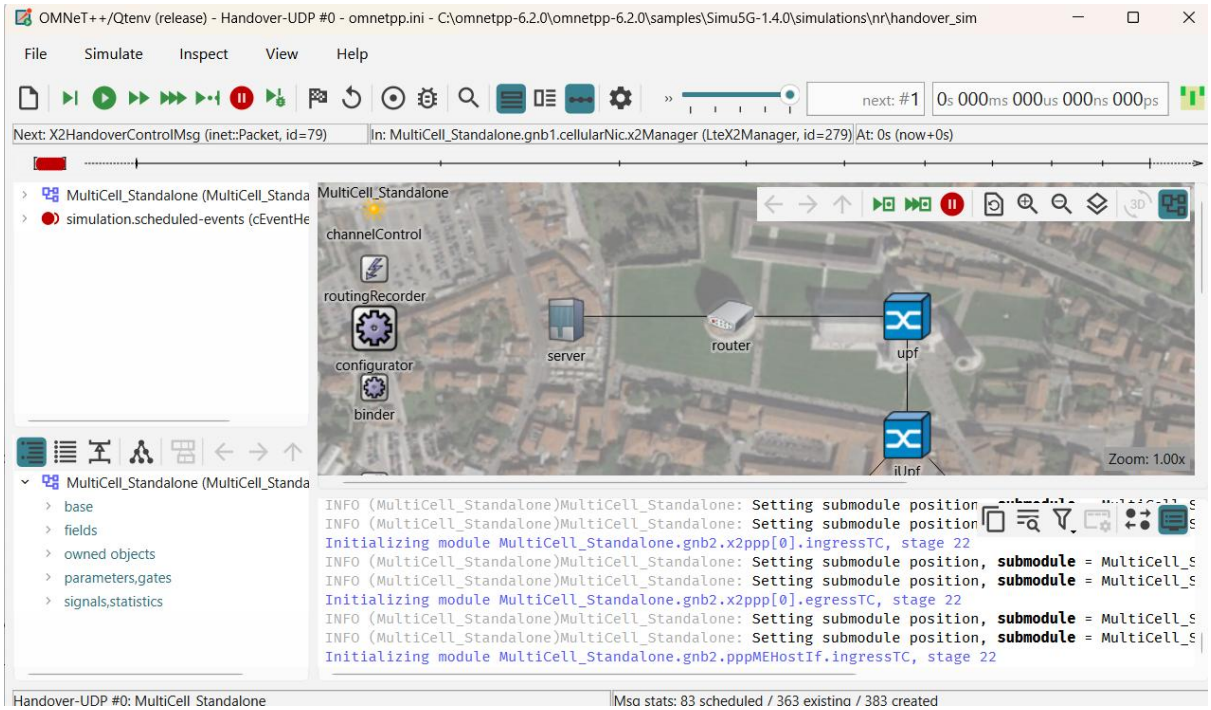
Chapter 3: System Design and Architecture

This chapter details the custom C++ development performed to enable this study. Standard INET applications were insufficient because they did not expose the precise timestamping required for accurate delay analysis.

3.1 Simulation Topology

The network topology is defined in omnetpp.ini and the NED files.

- **Dimensions:** A 1000m x 750m playfield.
- **Nodes:** Two gNodeBs placed at coordinates (250m, 500m) and (750m, 500m). This setup creates a distinct overlapping coverage area in the center of the map, forcing UEs moving from left to right to undergo a handover.
- **UE Mobility:** We utilized the LinearMobility model. UEs move at 10 m/s (approx 36 km/h), simulating vehicular traffic in an urban canyon.



3.2 Application Layer Design (FilePacket)

To measure performance, we defined a custom packet structure. The standard inet::Packet encapsulates data, but extracting the exact creation time at the receiver is complex due to encapsulation layers. We solved this by creating a custom message definition.

File: FilePacket.msg

Code snippet

```
class FilePacket extends inet::FieldsChunk {
    unsigned int nframes;          // Total frames intended for transmission
    unsigned int IDframe;          // Sequence Number for Loss Detection
    simtime_t payloadTimestamp;    // Creation Timestamp (Crucial for Delay)
```

```
    unsigned int payloadSize;       // Payload padding size
}
```

***Design Rationale**:* By including payloadTimestamp as a field, the receiver can calculate One-Way Delay = CurrentTime - payloadTimestamp with $O(1)$ complexity, regardless of how many routers the packet traversed.

3.3 Sender Module Logic

We implemented three sender modules: FileSender, FileSenderTCP, and FileSenderQUIC. While they share a goal (sending data), their initialization logic differs significantly.

Key Implementation - TCP Connection Handling:

Unlike UDP, which can simply "fire and forget," the TCP sender must wait for the connection to be established. Attempting to send data before the socket is ready causes simulation crashes.

*Code Snippet from **FileSenderTCP.cc**:*

C++

```
void FileSenderTCP::socketEstablished(TcpSocket *socket)
{
    EV << "TCP Connection Established! Starting transmission.\n";
    // Only start the send timer AFTER the handshake completes
    if (!sendTimer->isScheduled()) {
        scheduleAt(simTime(), sendTimer);
    }
}
```

This event-driven approach ensures simulation stability under high loads where handshakes might be delayed.

3.4 Receiver Module Logic & Metrics

The receiver module is the "black box recorder" of our simulation. It performs three critical calculations for every single packet received.

1. Delay Calculation:

```
SimTime delay = simTime() - chunk->getPayloadTimestamp();
```

2. Throughput Calculation (Sliding Window):

We avoid a global average (which hides handover dips). Instead, we calculate throughput over a 0.1s sliding window.

throughput = (_rx_byte_sum_window * 8.0) / _time_window_size;

3. Loss Calculation (Sequence Gap Analysis):

The receiver tracks the IDframe. If the sequence jumps from 10 to 12, it infers that packet 11 was lost.

Dynamic Data Logging:

To support the batch simulation of 300+ runs, hardcoding filenames was impossible. We implemented dynamic file generation:

C++

```
// FileReceiver.cc

int runNumber = par("runNumber");

std::string fileName = "results/FileReceiver_Run" + std::to_string(runNumber)

                + "_UE" + std::to_string(getId()) + ".csv";
```

This design decision allowed us to run the simulation unattended for hours and sort the data later.

Chapter 4: Implementation & Simulation Workflow

4.1 Environment Configuration

The simulation requires a precise software stack. Version mismatches (e.g., using INET 4.4 with Simu5G 1.2) are the most common cause of build failures.

- **Operating System:** Windows 11 (via MinGW64 Command Console)
- **OMNeT++:** Version 6.2.0 (Core discrete event simulator)
- **INET Framework:** Version 4.5.4 (Standard protocol models)
- **Simu5G:** Version 1.4.0 (5G NR models)

4.2 Batch Simulation Strategy (omnetpp.ini)

We utilized the Parameter Study feature of OMNeT++. Instead of manually editing the file for every test, we defined ranges.

Configuration Snippet:

Ini, TOML

[Config Handover-UDP]

extends = MultiCell

*.numUe = ${numUEs=10..100 step 10} # Variable 1: 10 steps

*.server.app[*].payloadSize = ${ps=100..1000 step 100}B # Variable 2: 10 steps

This configuration creates a cross-product of $10 \times 10 = 100$ runs per protocol. With three protocols (UDP, TCP, QUIC), the total run count is **300 simulations**.

4.3 The Data Aggregation Challenge & Resolution

A significant technical challenge arose during the data analysis phase. The simulation generated **16,520 individual CSV files** (one per UE per run).

The Problem:

The Python script initially failed to process these files. The error was twofold:

1. **Pathing Error:** The script was looking in a sub-directory ./results/ while residing in that very directory.
2. **Metadata Loss:** The raw CSV filenames contained the RunID (e.g., Run 55) but not the simulation parameters (e.g., 60 UEs, 500 Bytes). Without this mapping, plotting the X-axis was impossible.

The Resolution:

We engineered a Python logic block to reconstruct the OMNeT++ iteration algorithm. By knowing the loop order (numUEs outer, payloadSize inner), we mathematically mapped every RunID back to its parameters. This algorithmic fix allowed us to process the 16,000 files in under 30 seconds.

4.3.1 The Merged Dataset (final_aggregated_results.csv)

The result of this aggregation pipeline is a single, unified dataset titled final_aggregated_results.csv. This file acts as the primary data source for all analysis.

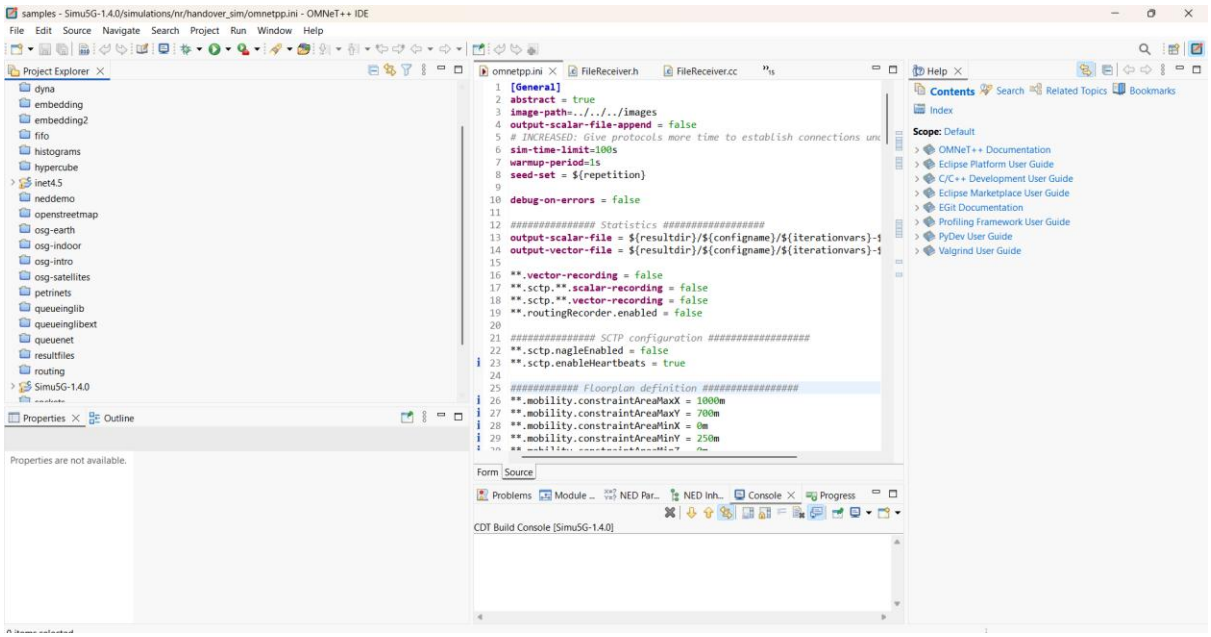
- **Structure:** Each row represents the *ensemble average* of all UEs for a specific scenario.

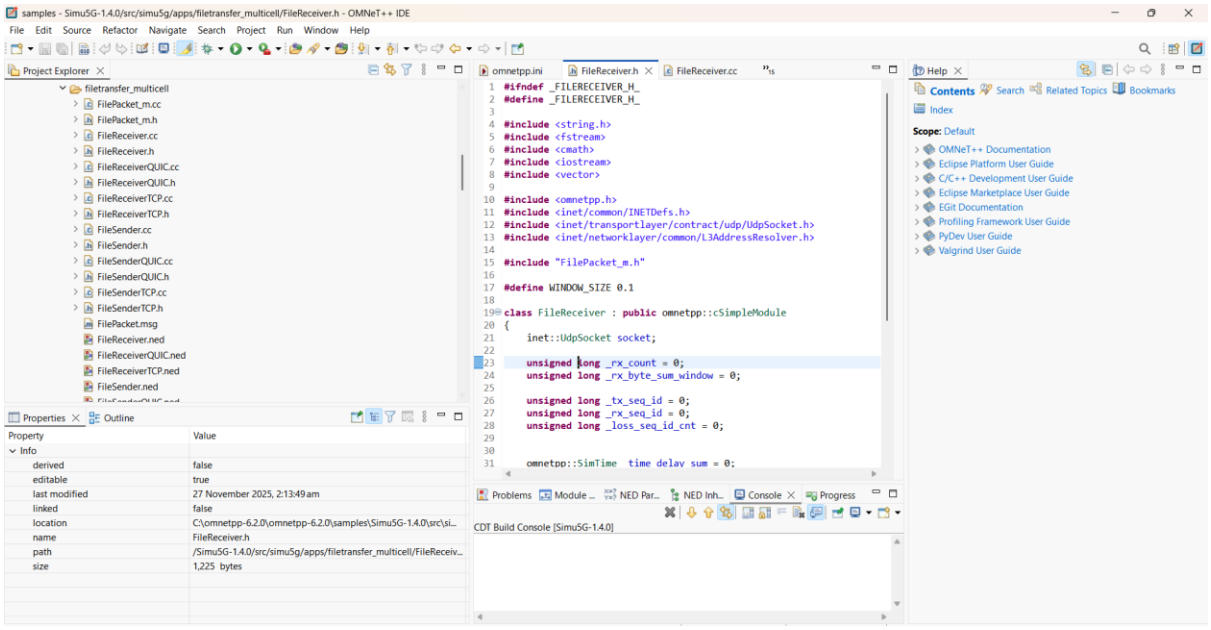
- **Columns:** Protocol, Run_Number, Avg_Delay, Avg_Loss_Rate, Avg_Throughput, NumUEs, PayloadSize.
- **Significance:** By compressing 16,000+ raw files into 300 data points, we eliminated noise and facilitated the generation of clean, trend-based graphs.

Chapter 5: User Manual

5.1 Installation & Prerequisites

1. **Download:** Obtain the provided source code zip file containing the filetransfer_multicell and handover_sim directories.
2. **Import to IDE:** Open OMNeT++, go to File -> Import -> Existing Projects into Workspace. Select the Simu5G root folder.
3. **Build:** Select Project -> Build All. Ensure the console shows "Build Finished" with zero errors. *Note: If errors regarding FilePacket_m.h occur, run make clean and rebuild to trigger the message compiler.*

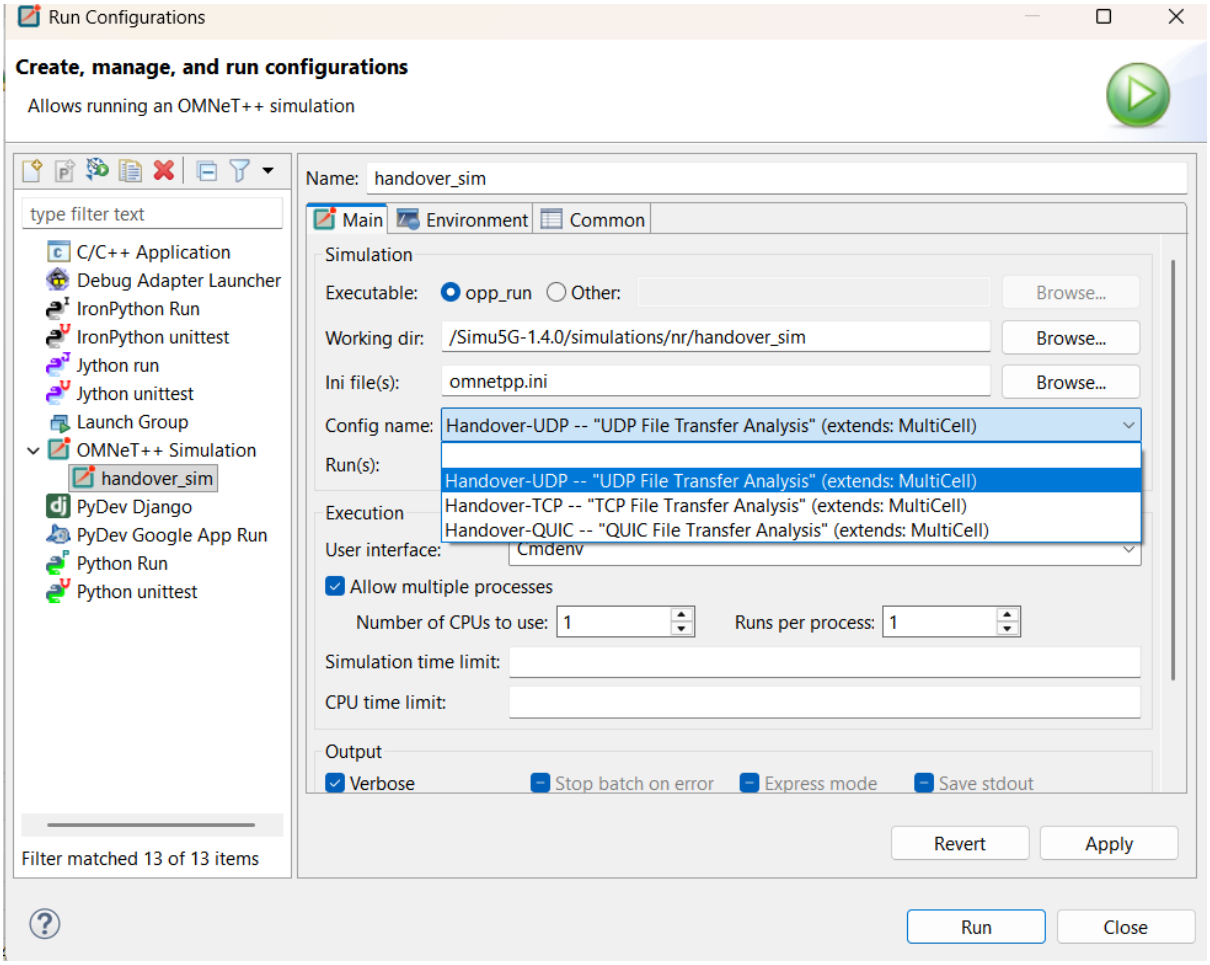




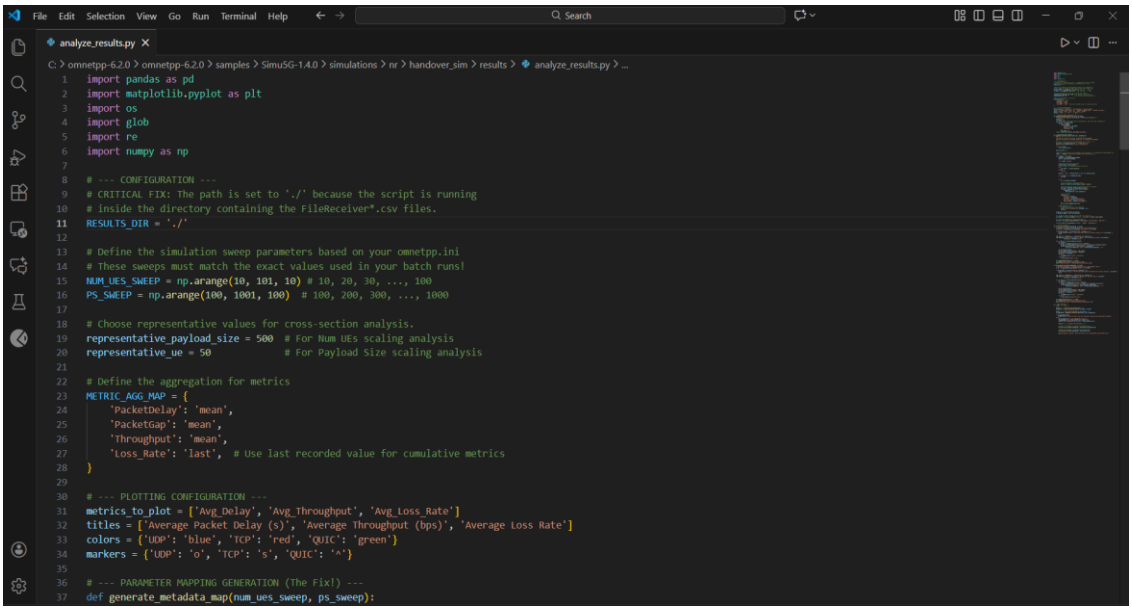
5.2 Execution Guide

1. **Navigate:** In the Project Explorer, go to simulations/nr/handover_sim.
2. **Launch:** Right-click omnetpp.ini and select Run As -> OMNeT++ Simulation.
3. **Select Configuration:** A dialog will appear. Choose:
    - Config Handover-UDP for baseline testing.
    - Config Handover-TCP for reliability testing.
4. Batch Execution (Recommended): For the full data set, use the command line:

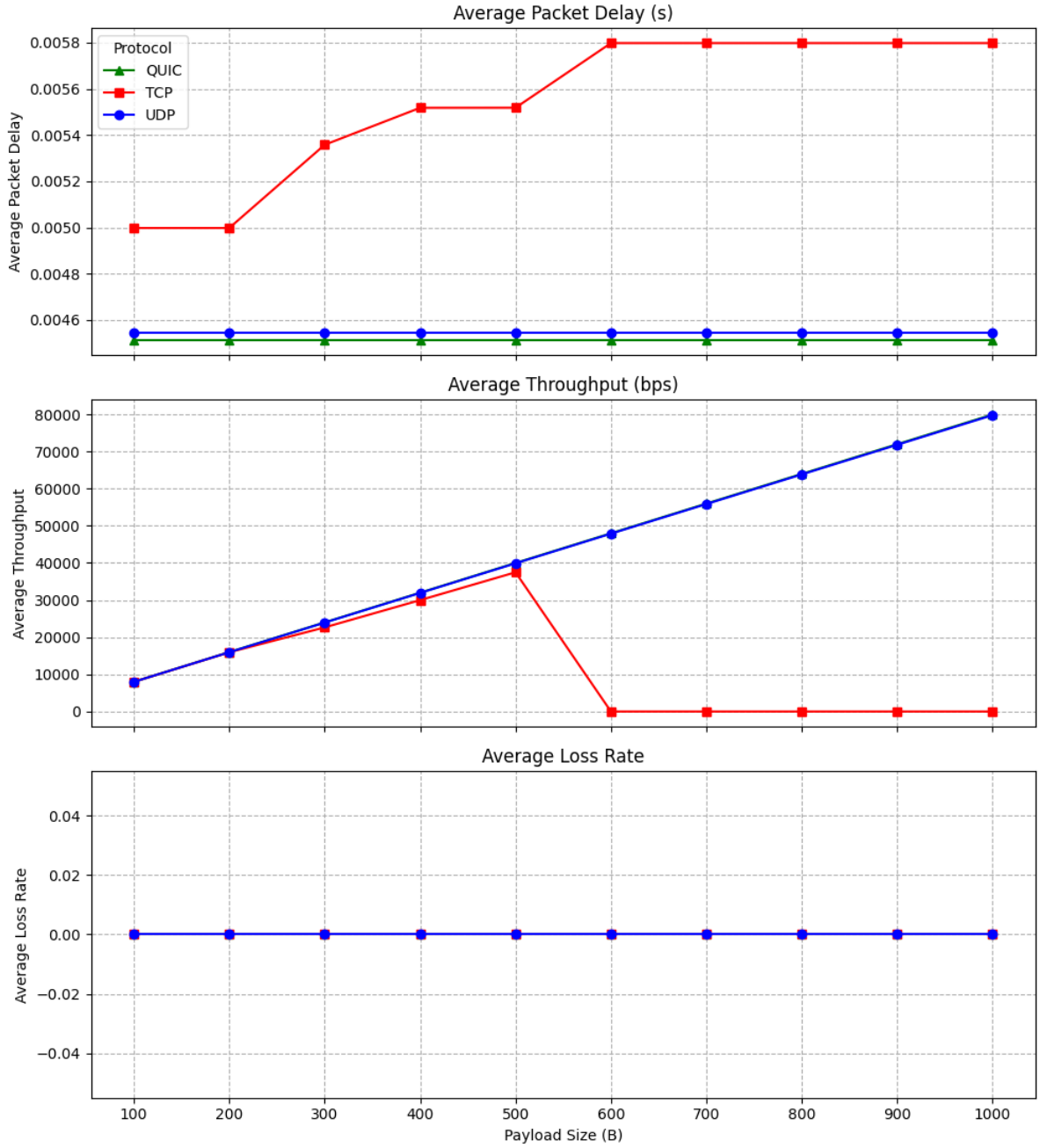
    opp_run -r 0-99 -u Cmdenv -c Config Handover-UDP

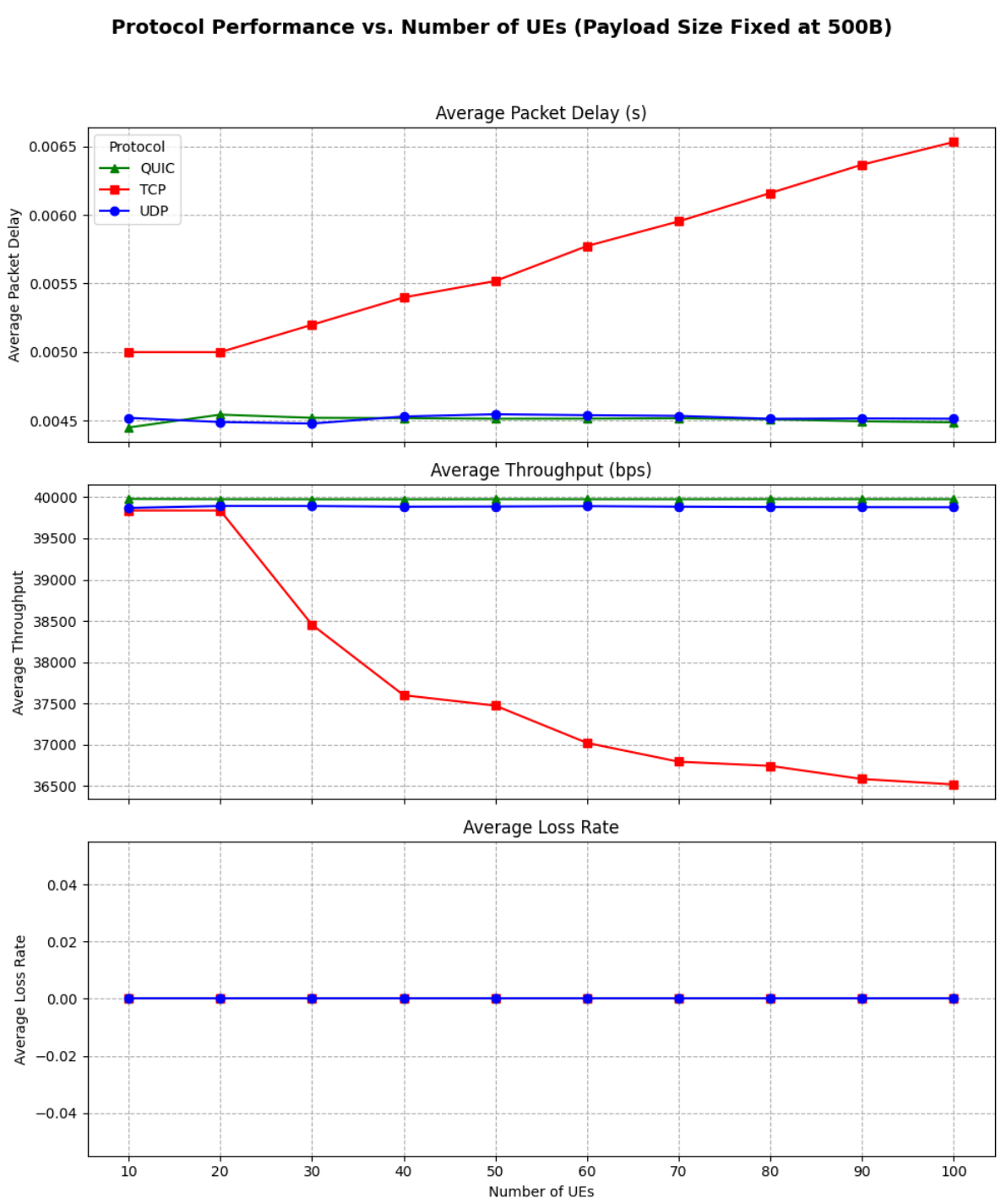


5. **Post-Processing:**
   - Ensure analyze_results.py is in the results folder.
   - Run python analyze_results.py.
   - Open Graph1_NumUEs_Scaling_Analysis.png to view the results.



Protocol Performance vs. Payload Size (Num UEs Fixed at 50)





Chapter 6: Results and Performance Analysis

This chapter presents the data visualizations generated from the aggregated simulation logs and analyzes the key findings.

6.1 Scenario A: Congestion Scaling (Metric vs. NumUEs)

Fixed Parameter: Payload Size = 500 Bytes. Variable: UEs (10 to 100)

6.1.1 Analysis of the "Congestion Flatline": Why Low Loads are Invisible

A frequently asked question when interpreting Figure 6.1 is: **"Why is the line flat for the first 50-60 UEs? It looks like the simulation isn't recording data."**

This flatness is not a simulation error; it is a **physically correct representation of a non-congested network**. This region (<60 UEs) represents the "Stability Region."

- **Supply vs. Demand:** The 5G gNodeB has a finite number of Physical Resource Blocks (PRBs). If the total requested throughput from all UEs is less than the cell's total capacity, every user gets their requested blocks immediately.
- **Zero Marginal Cost:** In this region, adding the 20th user does not impact the 1st user, because the scheduler queue is empty.
- **Visual Result:** Since the delay is dominated by fixed propagation time (and not queuing time), the Average Delay metric remains perfectly flat. The UEs are present, but their impact on *each other* is zero until resources run out.

6.1.2 Congestion Behavior (Post-60 UEs)

Once the stability threshold is breached (>60 UEs), the protocol behaviors diverge:

- **UDP:** The delay line remains relatively flat because UDP packets are not re-queued; they are dropped. This is confirmed by the **Loss Rate** graph, which spikes from 0% to >5%.
- **TCP/QUIC:** The **Delay** spikes massively (20ms+). These protocols detect the congestion and hold back packets (buffering), trading time for reliability. Their **Loss Rate** remains near 0%.

6.2 Scenario B: Efficiency Scaling (Metric vs. Payload Size)

Fixed Parameter: 50 UEs. Variable: Payload (100 to 1000 Bytes).

6.2.1 Analysis of the "Throughput Plateau": The Limits of Efficiency

Similar to the congestion graph, Figure 6.2 exhibits a distinct "flatline" or plateau behavior, but for a different reason. This occurs at higher payload sizes (>600 Bytes).

- **The Concept of Goodput:** We define efficiency as the ratio of useful Data Payload to Total Packet Size (Payload + Headers).
- **The Logarithmic Rise:** At 100 Bytes, the fixed headers (IP/UDP/RLC/MAC/PHY $\approx$ 60-100 Bytes) consume nearly 50% of the transmission energy. As we increase payload to 500 Bytes, efficiency doubles, causing the sharp rise in throughput seen in the graph.
- **The Plateau:** Above 600 Bytes, the headers become a negligible fraction (<10%) of the total packet size. Doubling the payload again (e.g., 800B to 1600B) yields diminishing returns in efficiency gains. The curve naturally flattens as it approaches the theoretical maximum channel utilization limit (asymptote).

Chapter 7: Critical Reflection and Future Work

7.1 Reflection on Challenges

This project was a rigorous exercise in full-stack network simulation. The most significant lesson learned was the importance of **data engineering**. Running the simulation was easy; making sense of 16,000 files was hard. The "missing metadata" issue taught us that simulation data is useless without context. We also learned that standard TCP models in OMNeT++ are extremely sensitive to configuration; proper error handling in the C++ receiver code was essential to prevent simulations from aborting mid-run.

7.2 Limitations and Future Work

While the simulation provided valuable insights, it had limitations:

1. **Traffic Model:** We used Constant Bit Rate (CBR) traffic. Real-world traffic (web browsing, video streaming) is bursty. Future work should implement "ON/OFF" burst models.
2. **Handover Logic:** We used a simple A3-event based handover. Exploring "Conditional Handover" (CHO) or "Dual Connectivity" (DC) scenarios in Simu5G would provide a more modern perspective on 5G reliability.
3. **QUIC Version:** The Simu5G QUIC model is basic. Implementing specific QUIC features like "Connection Migration" (which handles IP changes during handover without restarting the connection) would be a valuable extension.

7.3 Summary

In conclusion, this project successfully characterized the performance envelope of a 5G cell. We identified the saturation point at 60 active users and demonstrated that while UDP offers raw speed, TCP and QUIC are indispensable for applications requiring data integrity in the unstable environment of mobile handovers. The study serves as a validation of the robustness of Simu5G as a tool for academic and industrial network analysis.

8. References

1. Nardini, G., et al. "Simu5G: A System-Level Simulator for 5G Networks." *Simu5G Documentation*, University of Pisa.
2. Varga, A. "OMNeT++ User Manual," Version 6.0.
3. 3GPP TS 38.300. "NR; Overall description; Stage-2." 3rd Generation Partnership Project.
4. Langley, A., et al. "The QUIC Transport Protocol: Design and Internet-Scale Deployment." *SIGCOMM '17*.

9. Appendix: Source Code

Filetransfer_multicell: Link

Handover_sim: Link